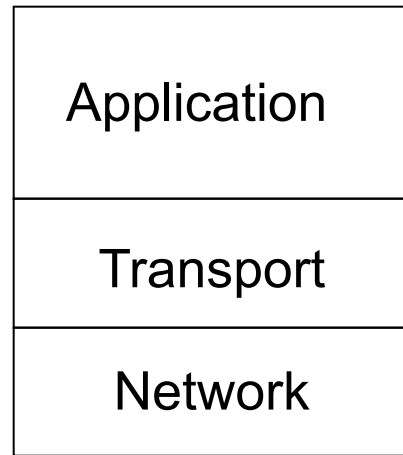
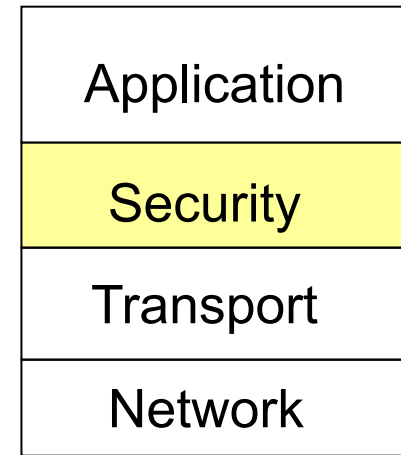


Transport Security

How should apps use security?



normal application



secure application

- Would be nice to provide an API to applications
- Useful to have security layer *above* transport

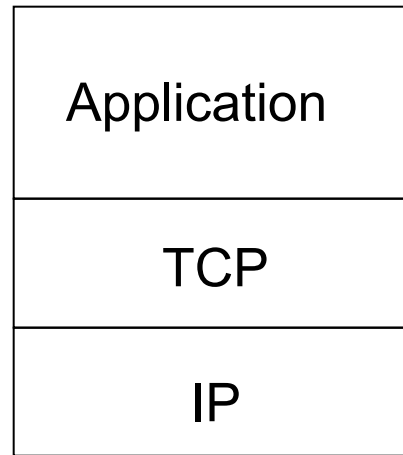
Transport Layer Security (TLS)

Providing security properties in a practical protocol

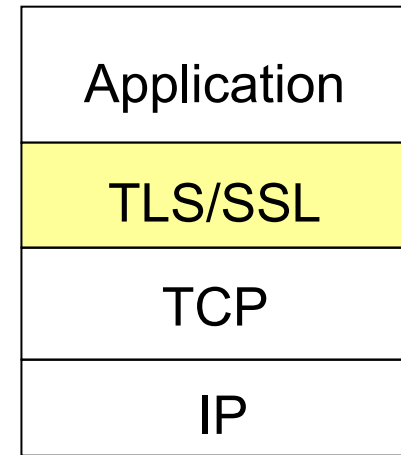
Goals of TLS

- Confidentiality
- Message integrity
- Server authentication (optionally, client authentication)
- Work in the context of the existing network protocol stack

TLS/SSL and the rest of the protocol stack



normal application



application with TLS/SSL

- TLS/SSL provides application programming interface (API) to applications
- C and Java libraries/classes readily available
 - Ex: OpenSSL

TLS/SSL “Cipher Suite”

- Cipher suite
 - public-key algorithm
 - symmetric encryption algorithm
 - Integrity hashing algorithm
- TLS/SSL supports several cipher suites
- **Negotiation**: client, server agree on cipher suite
 - client offers choices
 - server picks one

Common symmetric ciphers

- AES – Advanced Encryption Standard
- DES – Data Encryption Standard: block
- 3DES – Triple strength: block
- ChaCha: stream
- RC4 – Rivest Cipher 4: stream

SSL Public key encryption

- RSA with DH

Integrity hashing algorithms

- HMAC-MD5 and others

Handshake with Negotiation

1. server authentication
2. negotiation: agree on cipher suite
3. establish necessary keys
4. client authentication (optional)

TLS 1.2: All this takes a few round trip times to accomplish!

A typical handshake with RSA

Server
authenticated

Client created
session keys

Server created
session keys

Client hello
Client nonce, ciphers, TLS ver

Server hello
Server nonce, certificate, cipher choice
Client nonce encrypted with pvt key

Client premaster secret
Encrypted with server pub key

Server finished
Msg encrypted with sess key

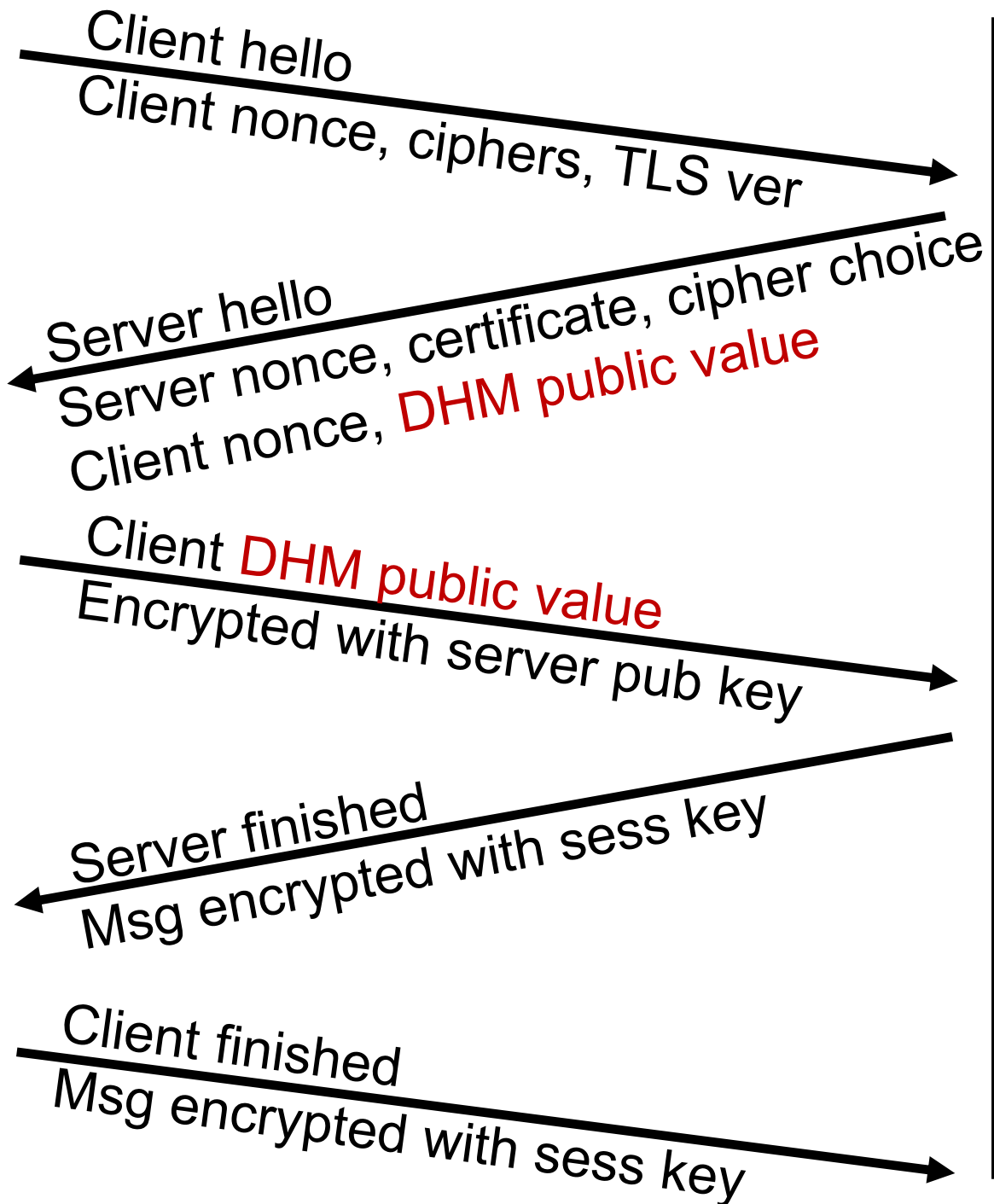
Client finished
Msg encrypted with sess key

A typical handshake with DHM

Server authenticated

Client created session keys

Server created session keys



QUIC handshake

Adding security in 1 (or 0) RTT to your transport connection

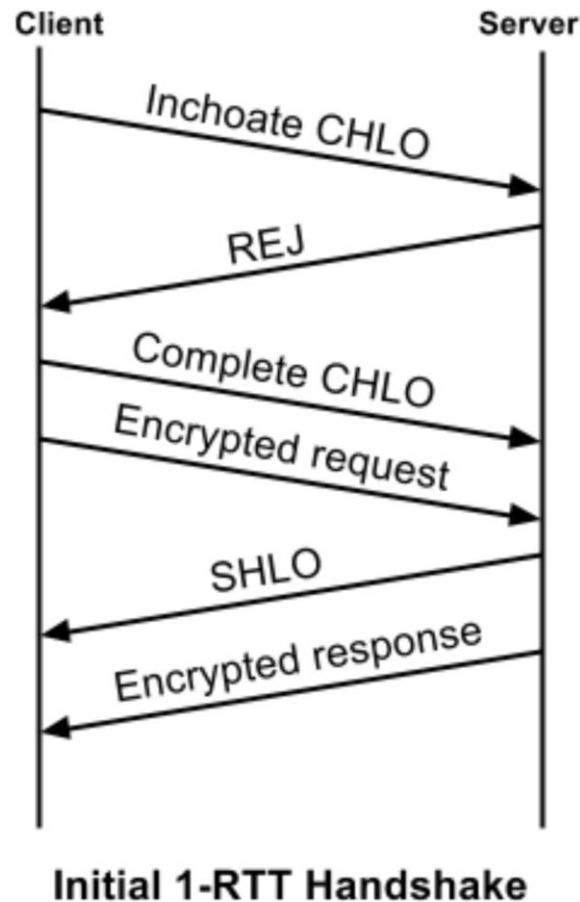
Optimize the common case

Goals of QUIC handshake

- Reduce round-trip times to get set up with secure connection
- Initial handshake takes 1 RTT, starting from the first byte received from the client, before server can send data
- Later, server can send data immediately upon receiving the first byte from the client (“0-RTT” handshake)

Initial 1-RTT handshake

- Client first sends an “inchoate” (incomplete) client hello message



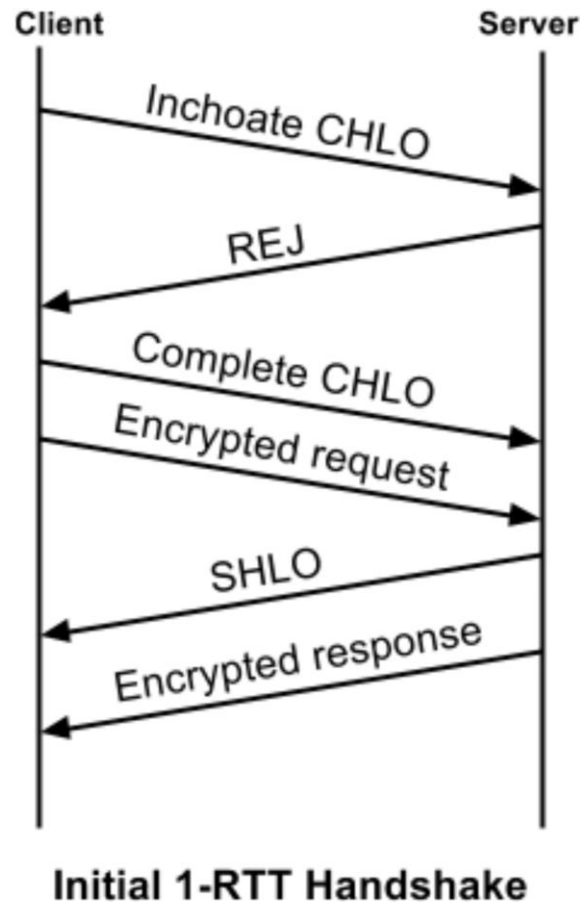
Server REJ message contains:

- (1) server's long-term DHM public value
- (2) Certificate chain authenticating the server
- (3) Signature of DHM public value using the certified (private) key
- (4) Client source address token signed by server

What properties does each of these (help) establish?

Initial 1-RTT handshake

- Client first sends an “inchoate” (incomplete) client hello message



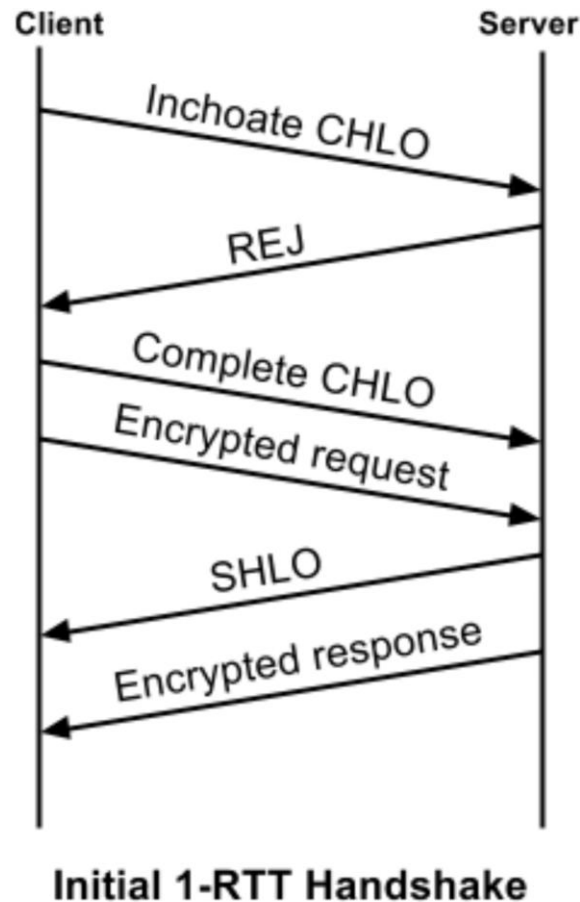
Server REJ message contains:

- (1) server's long-term DHM public value
- (2) Certificate chain authenticating the server
- (3) Signature of DHM public value using the certified (private) key
- (4) Client source address token signed by server: **session ticket**

Useful later on to authenticate the client
Prevent DoS (of server and other clients)

Initial 1-RTT handshake

- Finish the handshake



Client sends its ephemeral DHM public value to server in a complete CHLO.

Server then responds with its ephemeral DHM public value.

Ephemeral DH values provide **forward secrecy** for the communication

Other handshake details

- In later handshakes, client optimistically sends its source address token to the server
- Optimistic version negotiation: client proposes a cipher suite and encrypts first request with that cipher suite
 - If server can't speak that cipher suite, it forces version renegotiation, similar to TLS
- Both techniques optimize the common case when clients speak to a known server with an agreed-upon cipher suite

Important principle

- Use a hash of all the plaintext data that was exchanged in the handshake when generating the symmetric key for the connection
 - True in both TLS and QUIC
- Why?
- Example: cipher **downgrade** attacks
 - Provide “backward integrity” over all prior messages

